

Exercise - 6 Solutions

6.1: Why Explainability?

What are the advantages of explainable models in a safety-critical context?
What are potential disadvantages or limitations of current explainability methods?

→

Explainability is useful because it helps us understand **why** a model made a certain decision. In safety-critical systems, this is important because we cannot only trust high accuracy; we also need to know whether the model is using the right evidence.

As mentioned in slides - explainability is useful for certification, fairness, bias mitigation, and understanding errors.

Advantage	Explanation
Debugging model errors	Explanations can show whether the model looked at the correct object or at irrelevant background features.
Finding bias	If a model focuses on wrong features, such as background, weather, or camera artifacts, explainability can reveal this.
Supporting safety cases	In certification, we need evidence that the system behaves safely. Explanations can support this evidence.
Improving trust and transparency	Engineers, auditors, and users can better understand model behaviour.

→

Explanations are **evidence, not guarantees**, and that a heatmap is not automatically a faithful explanation.

Current explainability methods also have limitations - Explanations can look plausible but still be unfaithful to the real decision process. They may highlight correlated features instead of causal features, and different explanation methods may disagree. There is also a risk that users overtrust a model because the explanation looks convincing. Therefore, explainability should be treated as supporting evidence, not as a formal proof of safety.

6.2: Local vs. Global Explainability

Describe the difference between local and global explainability. Name one method for each kind and explain what question it answers.

→

Global explainability is understanding the decision process itself, while **local explainability** means understanding the decision process for a single input.

Type	Meaning	Main question
Local explainability	Explains one specific prediction for one specific input.	“Why did the model make this decision for this image?”
Global explainability	Explains the general behaviour of the model.	“What has the model learned overall?”

Local explainability example: Grad-CAM →

For one image, Grad-CAM creates a heatmap showing which parts of the image were important for the predicted class.

For example, in a pedestrian detector, Grad-CAM can answer - Did the model focus on the pedestrian, or did it focus on the road/background?

This is local because it explains one prediction for one image.

Global explainability example: Filter visualization →

Filter visualization tries to understand what a CNN filter has learned by finding the input pattern that maximally activates that filter.

It answers - Does this filter detect edges, textures, shapes, or object parts?

This is global because it helps us understand the model's general internal behaviour, not only one image.

6.3: Saliency vs. Occlusion

Briefly describe the saliency method and the occlusion method. What is one advantage and one disadvantage of each compared to the other?

→

Saliency method

The **saliency method** explains a prediction by checking how sensitive the model output is to each input pixel.

It calculates the gradient of the class score with respect to the input image pixels:

$$\frac{\partial \hat{y}_c}{\partial x_{ij}}$$

In simple words:

If a small change in a pixel strongly changes the prediction, that pixel is considered important.

For an image classifier, saliency produces a heatmap showing which pixels most influenced the prediction. **Saliency maps - measures prediction sensitivity using derivatives with respect to input pixels.**

Occlusion method

The **occlusion method** explains a prediction by hiding parts of the image and observing how the prediction changes.

Steps:

Step	Description
1	Select a class, for example "pedestrian"
2	Cover one part of the image

3	Measure how much the class probability changes
4	Repeat this for all image regions

If covering a region causes the pedestrian probability to drop strongly, that region was important for the prediction. Occlusion is a general approach that can be applied to different models.

Comparison →

Method	Advantage	Disadvantage
Saliency	Fast, because it uses gradients from one backward pass	Can be noisy and hard to interpret
Occlusion	Easier to understand because it directly tests what happens when an image region is removed	Slower and computationally expensive, especially for high-resolution maps

6.4: Chain-of-Thought Fidelity

Modern LLMs often produce a thinking trace (chain of thought) alongside their answer.

1. What does it mean for a thinking trace to be faithful? Why is faithfulness hard to verify?

→

A thinking trace is **faithful** if it honestly reflects the real reason why the model generated its answer. Simply, **the explanation should match the actual internal reasoning that caused the answer.**

For example, if an LLM says: “I rejected the request because it violates the safety policy,”

then that should be the real reason, not just a nice-looking explanation written after the answer was already chosen.

→ **ASK - whether chain-of-thought really reflects the underlying computation.**

Faithfulness is hard to verify because we usually cannot directly observe the real internal computation of the model.

The model may produce a convincing rationale, but that rationale may not be the actual cause of the answer.

2. What is simulatability as a measure of thinking-trace quality? Give an example scenario where simulatability is satisfied but the trace is still unfaithful.

→

Simulatability means: Given the thinking trace, can an outside observer predict the model's final answer?

If a human reads the chain of thought and can correctly guess the answer, then the trace is simulatable.

Example→

Thinking trace: A pedestrian is visible in front of the car. The distance is short. Continuing would be dangerous.

Question: Should the vehicle brake?

Answer: Brake. Here, a human can predict the answer from the trace, so it is simulatable.

But simulatability does **not guarantee faithfulness**.

Example where simulatability is satisfied but trace is unfaithful -

Suppose an LLM answers a loan application:

Thinking trace: The applicant has low income and unstable employment, so the loan should be rejected.

Answer: Reject loan.

A human can predict the answer from the trace, so simulatability is satisfied.

But the real hidden reason may be - the model used a biased feature such as postal code or ethnicity-correlated information. In that case, the trace is understandable but not faithful.

→ The model may write a rationale that strongly hints at the answer while the true causal reason is different.

3. What does counterfactual simulatability add? Why is it a stricter test?

→

Counterfactual simulatability adds a stronger test.

Instead of only asking:

“Can we predict the answer from the trace?”

it asks something like:

If the question or answer were changed in a related case, would the trace still correctly explain the changed answer?

It checks whether the reasoning trace contains information that truly supports the answer, rather than just sounding consistent.

→ It is stricter because it tests whether the explanation works under related counterfactual situations.

For example:

Case	Question	Trace	Answer
Original	“Should this loan application be approved?”	“The applicant has stable income, low debt, and a good repayment history.”	Approve
Counterfactual	“Should this loan application be rejected?”	The same trace should not support rejection, because stable income, low debt, and good repayment history point toward approval.	No

If the same style of trace can be used to justify different answers, then it is probably not faithful.

4. Why does an unfaithful thinking trace pose a safety risk? Give one concrete example.

→

An unfaithful thinking trace is risky because it can make users or auditors trust the model for the wrong reason.

The model may appear safe, careful, or policy-compliant, while its real decision process is flawed.

Example -

Imagine an LLM-based credit risk assistant says:

Thinking trace: The applicant has unstable income, a high debt-to-income ratio, and a short credit history.

Answer: Reject loan.

This explanation sounds reasonable because income stability, debt level, and credit history are valid factors in credit assessment.

But in reality, the model may have mainly relied on an unfair shortcut, such as the applicant's postal code, neighborhood, surname, or another feature correlated with protected attributes.

This is dangerous because we may believe the model rejected the loan for legitimate financial reasons, while the real decision process may be discriminatory. The explanation hides the true failure, so the bank may keep using an unfair model and wrongly reject many applicants.

6.5: Applying an Explainability Method

Choose one local explainability method from the lecture: occlusion, CAM, Grad-CAM, or saliency maps. Apply it to each of your three CARLA models.

1. Briefly describe how your chosen method works and why you chose it.

→ Used **Grad-CAM** as the local explainability method.

Grad-CAM creates a heatmap showing which image regions contributed most to the model's prediction. It uses the gradients of the prediction score with respect to the final convolutional feature maps. These gradients are used as weights to highlight important image regions.

I chose Grad-CAM because our CARLA models are based on **ResNet-18 CNNs**, and Grad-CAM is suitable for CNN models. It is also better than basic CAM because it does not require a special architecture with global average pooling directly before the classifier. Grad-CAM is more generally applicable to CNNs because it uses gradients instead of only classifier weights.

2. Select five correctly classified images - at least one per model. Visualize the explanation (heatmap or sensitivity map) overlaid on the original image.

→

Model	True label	Prediction	Observation
Pedestrian Detector	0	0	The model correctly predicted that no pedestrian is present. However, the heatmap focuses on the left-side road/building area rather than a clear object.
Pedestrian Detector	0	0	The prediction is correct, and the heatmap is mostly weak. This is acceptable because there is no pedestrian object to highlight.
Traffic Light Detector	0	0	The model correctly predicted no traffic light. The heatmap is mostly weak and does not strongly focus on a traffic light.
Traffic Light Detector	1	1	The model correctly predicted traffic light present, but the heatmap mainly highlights road/vehicle-front or lower image regions instead of clearly focusing on the traffic light.
Vehicle Detector	0	0	The model correctly predicted no vehicle, but the heatmap highlights upper/background regions rather than a specific vehicle object.

3. Do the highlighted regions correspond to the relevant objects (e.g., does the map highlight the pedestrian when the model predicts “pedestrian present”)?

→

Only partly.

For some negative examples, the model correctly predicts that the object is absent, so it is not necessary for the heatmap to highlight a specific pedestrian, traffic light, or vehicle. In these cases, weak or scattered heatmaps are acceptable.

However, in the positive traffic light example, the model predicts **traffic light present** with high confidence, but the highlighted region does not clearly focus on the traffic light itself. Instead, the

heatmap appears to focus more on road or lower image regions. This suggests that the model may be using contextual or background features instead of only the actual traffic light.

So, the Grad-CAM results give mixed evidence. Some correct predictions seem reasonable, but some heatmaps suggest possible shortcut learning.

4. Select three misclassified images and apply the same method. Do the explanations give any insight into why the model failed?

→

Model	True label	Prediction	Probability	Observation
Pedestrian Detector	0	1	0.550	The model falsely predicts a pedestrian. The heatmap focuses on the center road/sidewalk area, possibly reacting to distant objects, shadows, or road structure.
Traffic Light Detector	0	1	0.898	The model falsely predicts a traffic light with high confidence. The heatmap strongly highlights the central road/building area, not a clear traffic light.
Vehicle Detector	0	1	1.000	The model falsely predicts a vehicle with very high confidence. The heatmap highlights the right-side fence/background region rather than a visible vehicle.

6.6: Explainability as a Diagnostic Tool

1. Suppose your explanation method reveals that your model predicts “pedestrian present” based primarily on a region of the sky rather than the pedestrian itself. What would this imply about the model’s generalization? What could cause this behavior?

→ Could be a Spurious feature.

If Grad-CAM shows that the model predicts **pedestrian present** because of a sky region instead of the pedestrian, this means the model is probably using a **spurious feature**.

A spurious feature is something that is correlated with the label in the training data, but is not actually the real cause.

What this implies about generalization – The model may work on the original test set but fail in new conditions.

For example, if the model learned:

bright sky / certain lighting pattern → pedestrian present

instead of → human body shape → pedestrian present

then it may fail when the sky color, lighting, weather, or town layout changes.

So the model has poor generalization because it is not learning the actual object. It is learning shortcuts from the training environment.

2. Now test your models on images from different conditions than the ones they were trained on - for example a different town layout, nighttime, or foggy weather. Apply the same explanation method as before.

(refer ipynb for implementation)

(a) Do the highlighted regions still correspond to the relevant objects?

→

Based on the Grad-CAM images, the highlighted regions **do not consistently correspond to the relevant objects**.

For the pedestrian detector, several shifted-condition examples show heatmaps focusing on:

Highlighted area	Observation
Road area	The model often focuses on road/sidewalk regions.
Buildings/background	In night and different-town examples, attention goes to scene structure.

Traffic light/background	In fog examples, the heatmap sometimes focuses around traffic-light or roadside regions instead of the pedestrian.
Weak/no clear object focus	Some correct negative predictions have almost no strong heatmap, which is acceptable because no pedestrian is present.

(b) Do you observe evidence that the model relied on background or spurious features (lighting, road texture, sky color) that are specific to the original training conditions?

→

Feature	Possible meaning
Foggy background	Model may be affected by weather/visibility.
Road texture	Model may rely on road layout instead of object shape.
Buildings	Model may learn town-specific scene patterns.
Traffic light region	Pedestrian detector may confuse context around intersections.
Bright/dark areas	Lighting changes may influence the model.

(c) How do the model's accuracy and explanation quality change across conditions?

→

Condition	Pedestrian Accuracy	Traffic Light Accuracy	Vehicle Accuracy
Original	0.7900	0.9367	0.8861
Fog	0.8056	0.5572	0.6556
Night	0.6506	0.2711	0.7603
Different Town	0.7578	0.7947	0.7606

Model	Accuracy trend
Pedestrian Detector	Accuracy is highest in fog, but drops at night and in different town.
Traffic Light Detector	Accuracy drops strongly in fog and almost collapses at night.
Vehicle Detector	Accuracy decreases under all shifted conditions, especially fog.